

📖 KARTA POMOCY — `models.py` w Django

❏ Po co jest model?

Model opisuje, **jakie dane mają być zapisane w bazie danych**.

📄 Przykład

```
from django.db import models

class Photo(models.Model):
    title = models.CharField(max_length=100)
    image = models.CharField(max_length=200)
    downloads = models.IntegerField(default=0)

    def __str__(self):
        return self.title
```

🔍 Jak to czytać?

`class Photo(models.Model):`
Tworzy model `Photo`, czyli wzór tabeli w bazie.

`title = models.CharField(max_length=100)`
Pole tekstowe, np. tytuł zdjęcia.

`image = models.CharField(max_length=200)`
Pole tekstowe, np. ścieżka do pliku `jpg/1.jpg`.

`downloads = models.IntegerField(default=0)`
Pole liczbowe — licznik pobrań.
Nowe zdjęcie zaczyna od 0.

`def __str__(self): return self.title`
Określa, jak obiekt ma się wyświetlać, np. w panelu admina.

⚠️ Ważna zasada

Model to opis danych, a nie same dane.

To znaczy:

- model tworzy strukturę tabeli,
- ale rekordy trzeba dopiero dodać.

Dlatego wpisywałeś **9 zdjęć przez admin** — bo galeria 3×3 potrzebowała **9 rekordów w bazie**.

🔄 Schemat działania

```
models.py
  ↓
opis tabeli
  ↓
makemigrations / migrate
  ↓
tabela w bazie
  ↓
dodanie rekordów
  ↓
wyświetlanie danych na stronie
```

❏ Zapamiętaj

`models.py` = projekt tabeli w bazie danych